😊 Azure Function

What is Azure Functions?

Azure Functions is a **serverless compute service** provided by Microsoft Azure that enables developers to execute small, event-driven pieces of code without managing infrastructure. It is designed to respond to various triggers, such as HTTP requests, timers, message queues, and changes in data sources.

As a **fully managed, scalable service**, Azure Functions automatically allocates computing resources as needed, making it ideal for lightweight, event-driven applications.

Key Benefits of Azure Functions

1. Serverless and Fully Managed

- Eliminates the need for infrastructure management, allowing developers to focus purely on writing code.
- Azure automatically handles scaling, monitoring, and security.

2. Cost-Efficient

- **Consumption Plan**: You only pay for the execution time of functions, reducing costs for infrequent workloads.
- **Premium and Dedicated Plans**: Suitable for high-performance and enterprise-level applications.

3. Event-Driven and Scalable

- Functions execute in response to various triggers (e.g., HTTP requests, database changes, event queues).
- Can scale **automatically** based on workload demand.

4. Multi-Language Support

- Supports **C#, JavaScript (Node.js), Python, Java, PowerShell, TypeScript, and F#**.
- Enables developers to choose their preferred language.

5. Rich Integration with Azure Services

- Works seamlessly with **Azure Blob Storage, Cosmos DB, Event Grid, Service Bus, SQL Database**, and more.
- Can be part of complex cloud-based workflows.

6. Supports Durable Functions

- Extends Azure Functions with **Durable Functions**, enabling **stateful workflows** and orchestrating multiple function executions.

7. Secure and Reliable

- Supports **Azure Active Directory (AAD) authentication**, Virtual Networks (VNETs), and managed identities.
- Fully monitored with **Azure Application Insights** for logging and performance tracking.

Use Cases of Azure Functions

1. API and Microservices

- Build **lightweight APIs** that handle HTTP requests without a full web server.
- Create **backend services** for mobile or web applications.

2. Background Task Processing

- Process **real-time data** asynchronously, such as image processing, file conversions, and data cleanup.
- Automate **cron jobs** with Azure Function **Timer Triggers**.

3. Event-Driven Automation

- **Trigger workflows** based on Azure Storage, Cosmos DB changes, or Service Bus messages.
- Automate tasks like **database updates**, notifications, or backups.

4. IoT and Real-Time Data Processing

- Process IoT data from **Azure IoT Hub** or **Event Hubs** in real-time.
- Analyze **sensor data**, detect anomalies, and trigger alerts.

5. Chatbots and AI Integration

- Integrate with **Azure Cognitive Services** for **chatbots, image recognition, and natural language processing (NLP)**.

6. CI/CD and DevOps Automation

- Automate **builds, deployments, and infrastructure provisioning** in a DevOps pipeline.
- Respond to repository changes with Azure DevOps or GitHub webhooks.

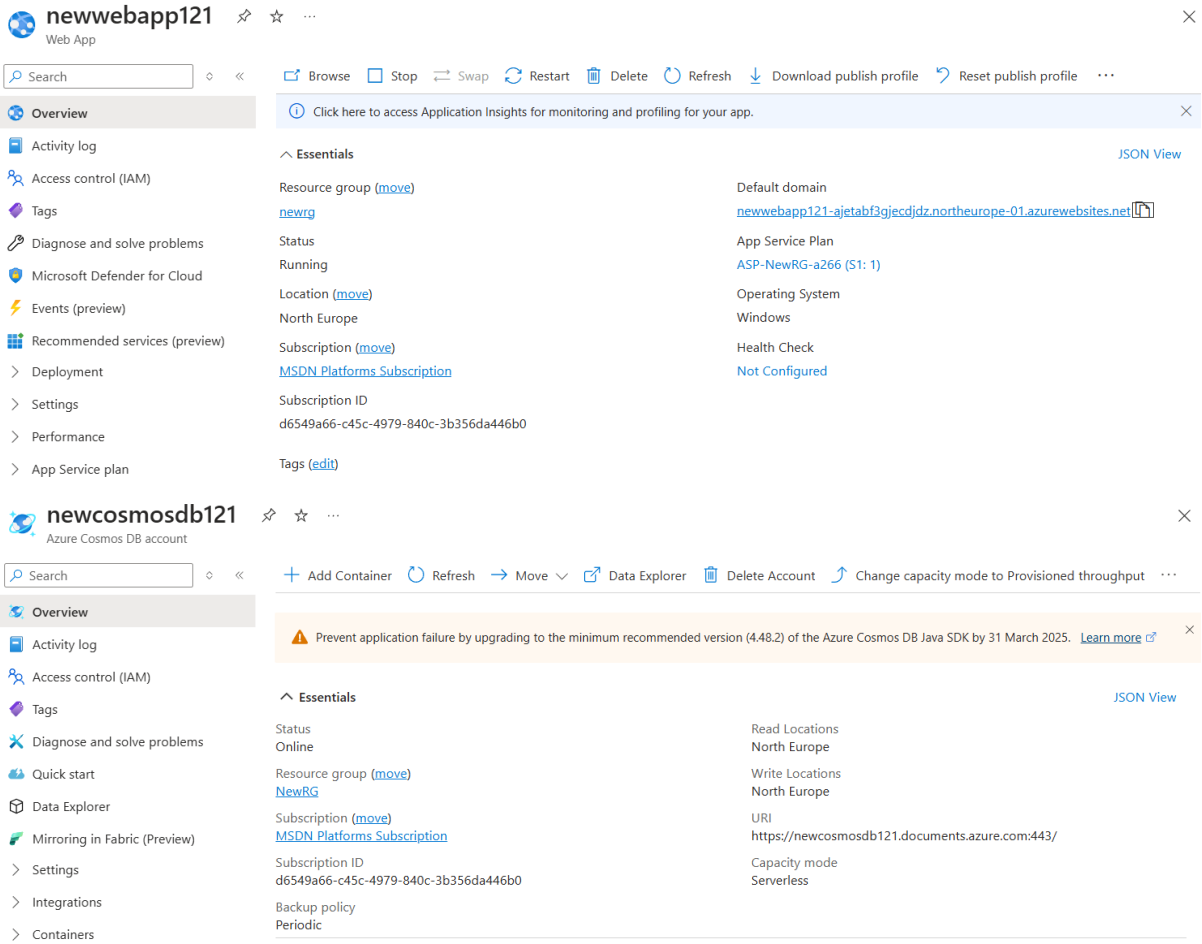
Conclusion

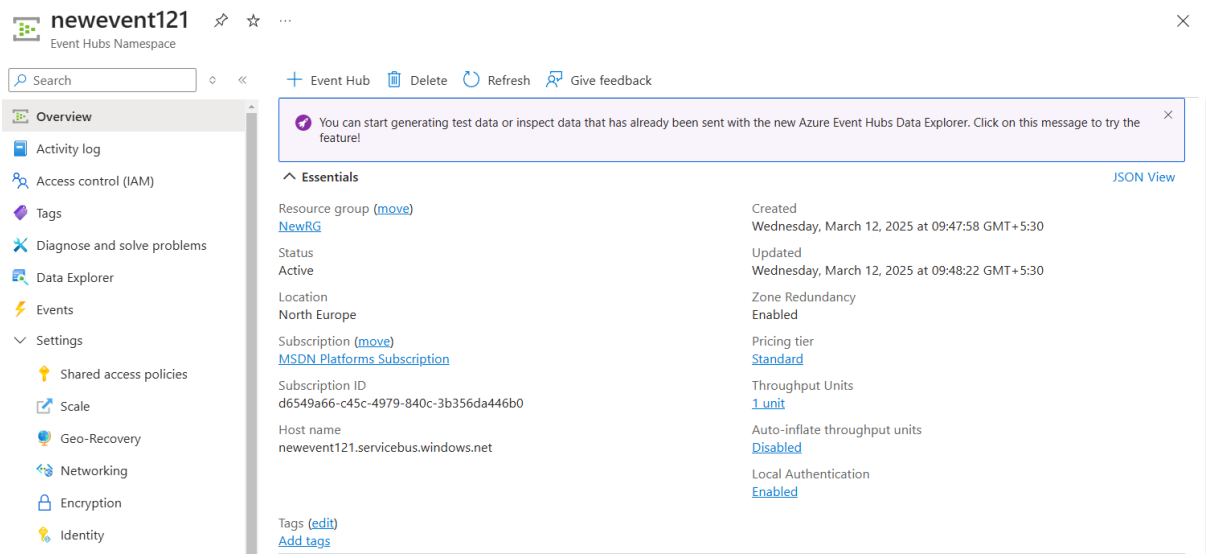
Azure Functions provides a **scalable, cost-effective, and event-driven** way to run cloud applications without managing servers. It is particularly useful for **API development, background processing, real-time automation, and serverless workflows**. With its **pay-per-use model** and **seamless integration with Azure services**, it is an excellent choice for modern cloud applications.

The process involves setting up an Azure Function App to process logs from an Azure Event Hub and store them in Azure Cosmos DB. First, create a Function App with an HTTP trigger, then configure a second function to listen to Event Hub events. Modify the connection strings in Visual Studio, update database details, and run the program locally. The function extracts relevant event data and writes it to Cosmos DB. Finally, publish the function to Azure, configure connection strings in the portal, and verify data ingestion. The end goal is an automated event processing pipeline for structured log storage.

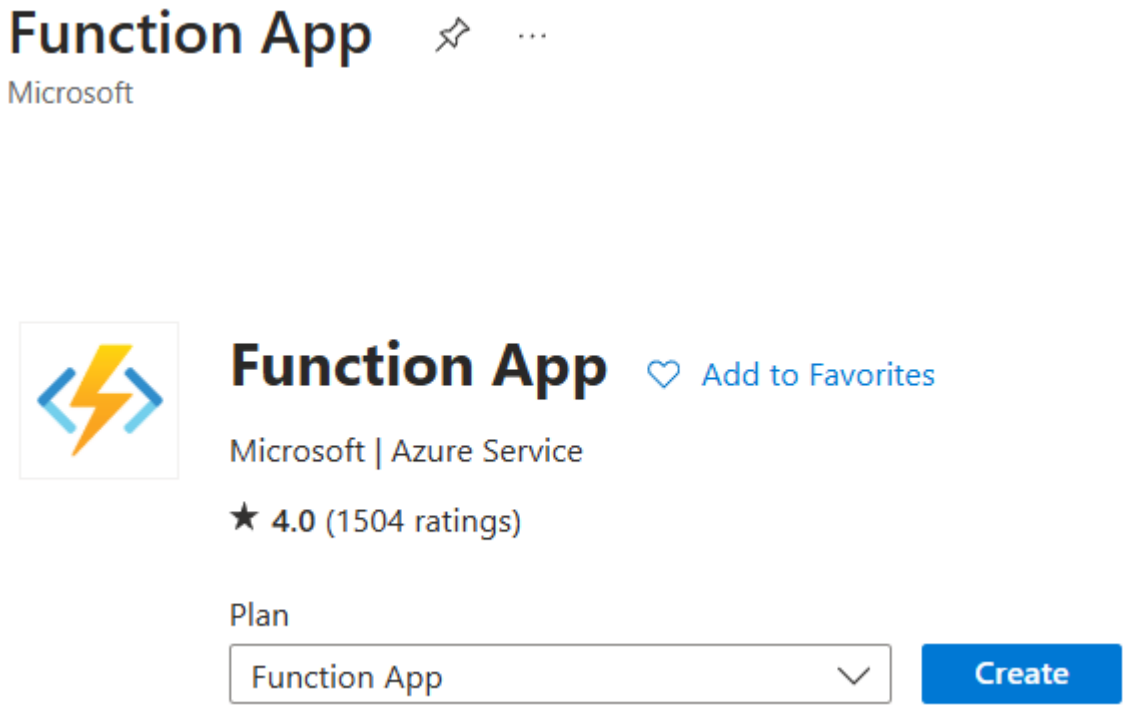
😄 To begin with the Lab

1. To work with this lab, you need to have resources beforehand. This is the prerequisite for this lab.
2. You should have a web app, a cosmos DB account, and an event hub in place. All of the resources are covered in previous labs.
3. Then download the zip files attached with this lab.

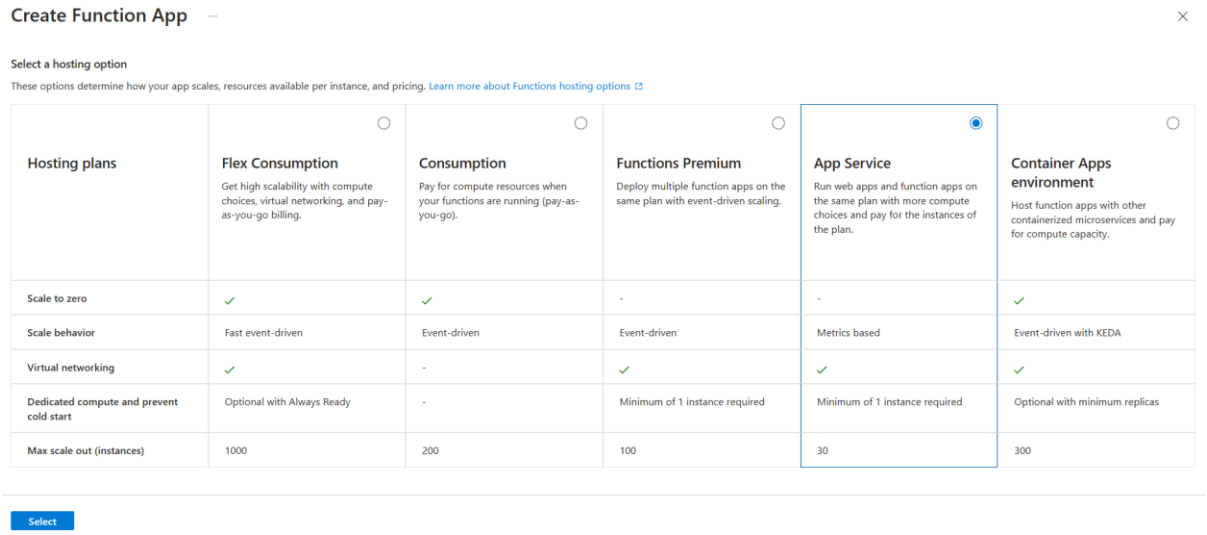




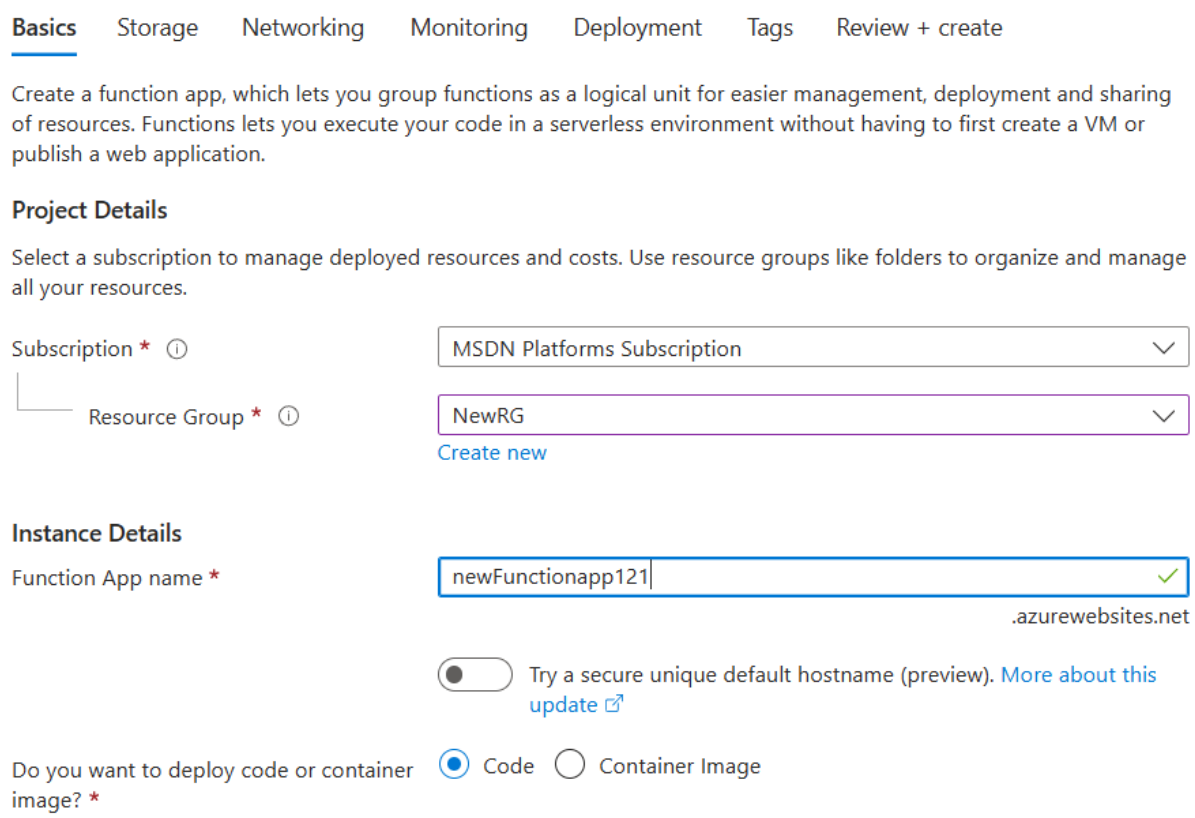
4. Now we need to create a function app. Click on Create.



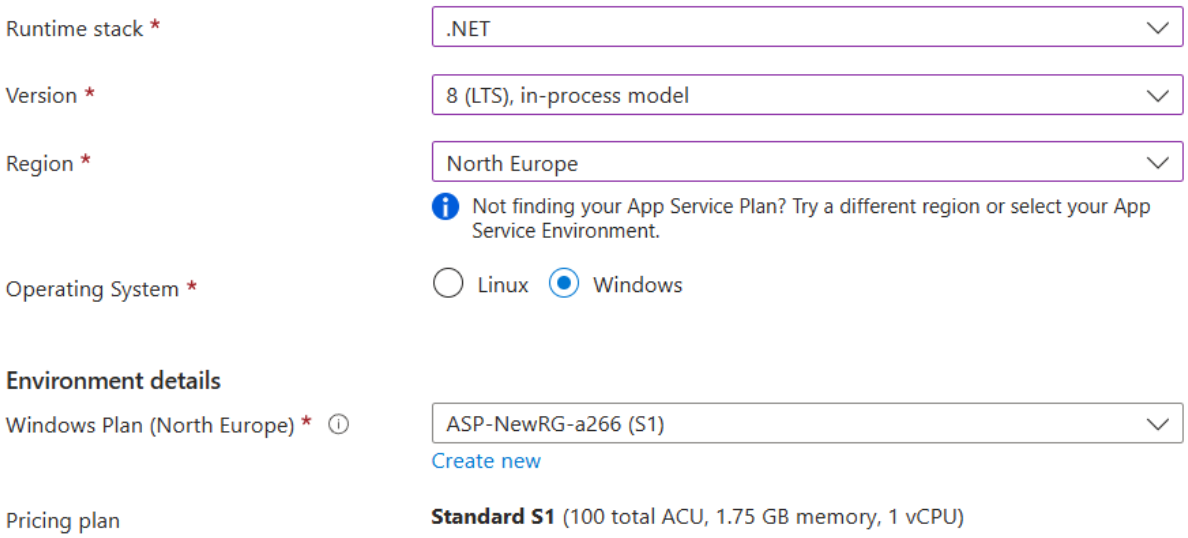
5. Then you have to choose App service as your hosting option. Click on Select.



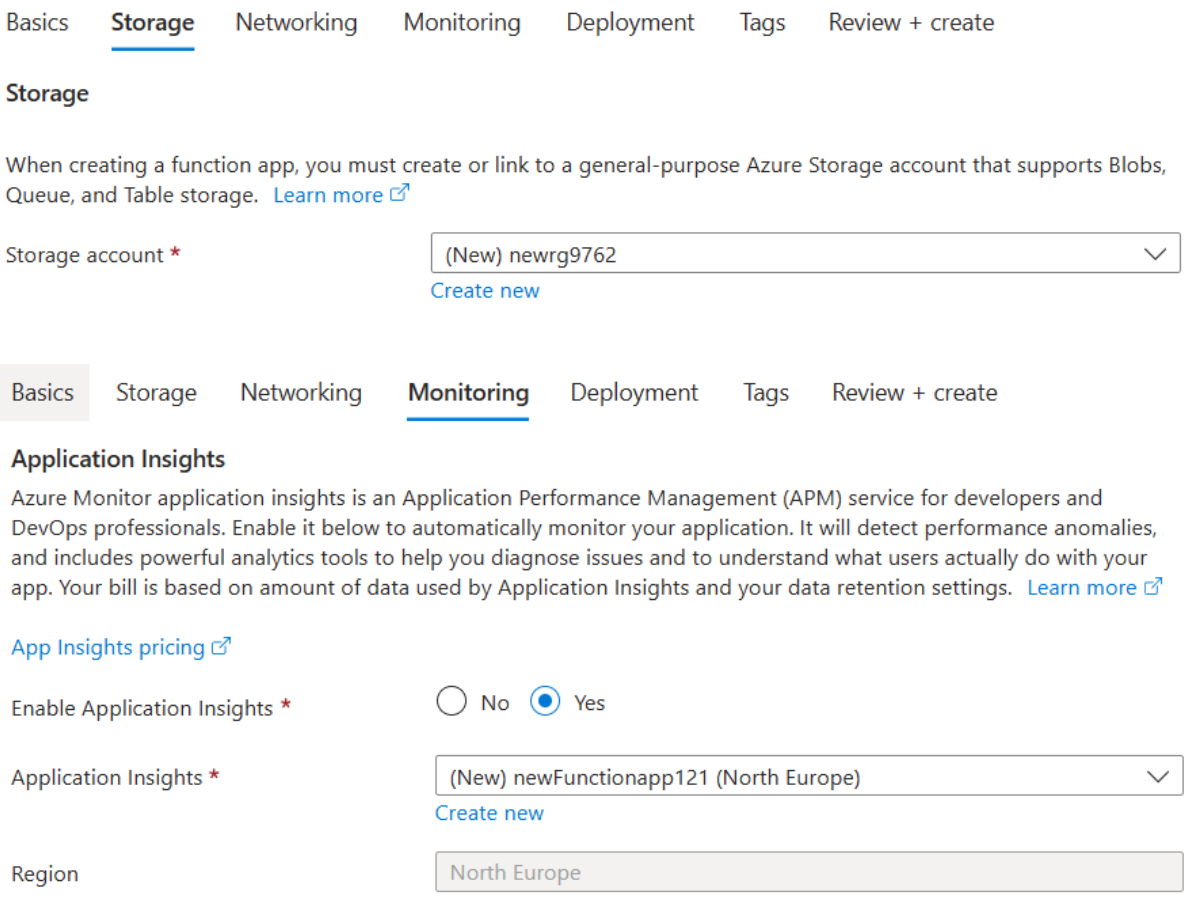
6. Now give it a name to your function and choose your resource group.



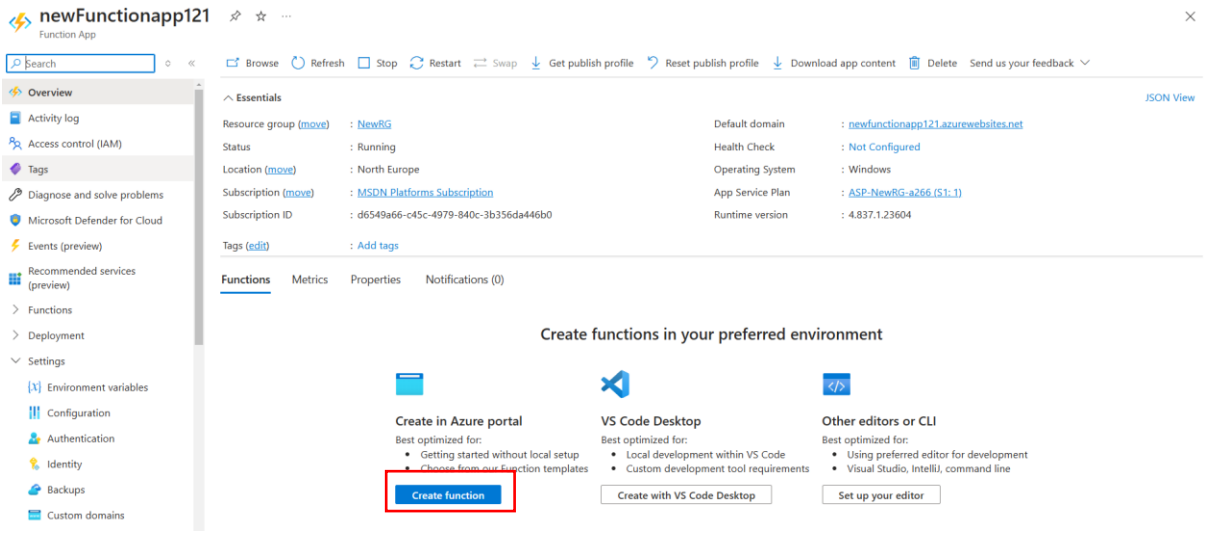
7. Choose the runtime stack as dot Net and choose your region.



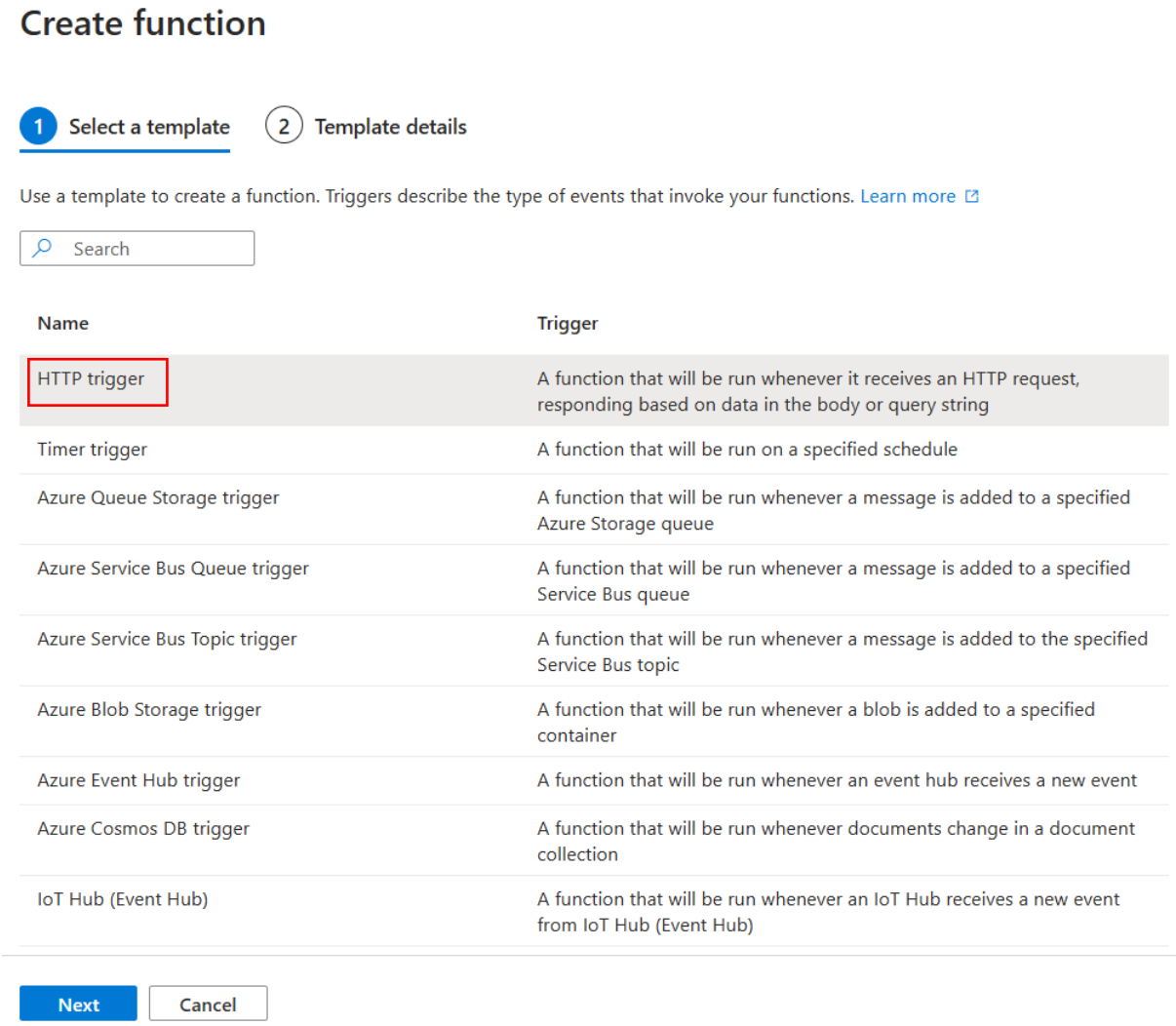
8. It will also create a storage account for you. Then in the monitoring section turn on the application insights feature.



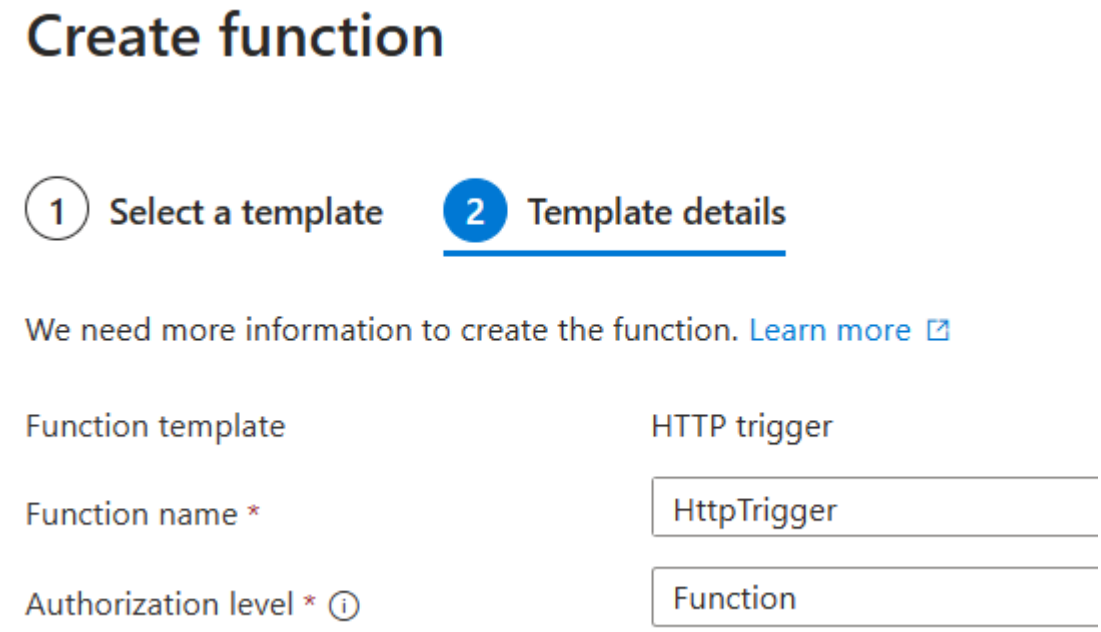
9. In your function app you can create a function. Click on Create.



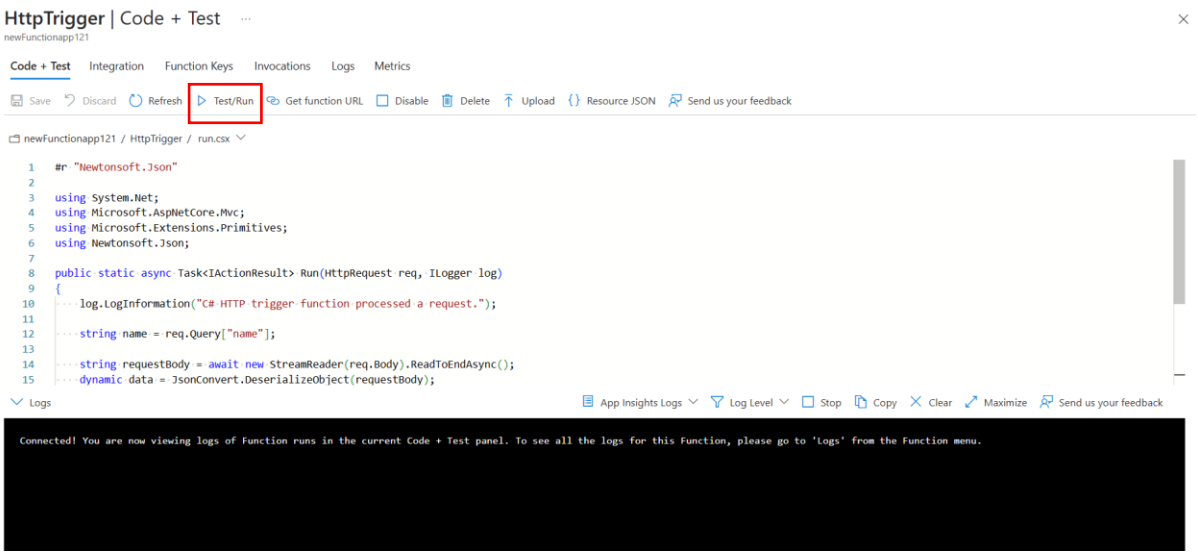
10. Here we have multiple functions to choose from, for now, to keep things simple we will choose HTTP trigger. Click on next.



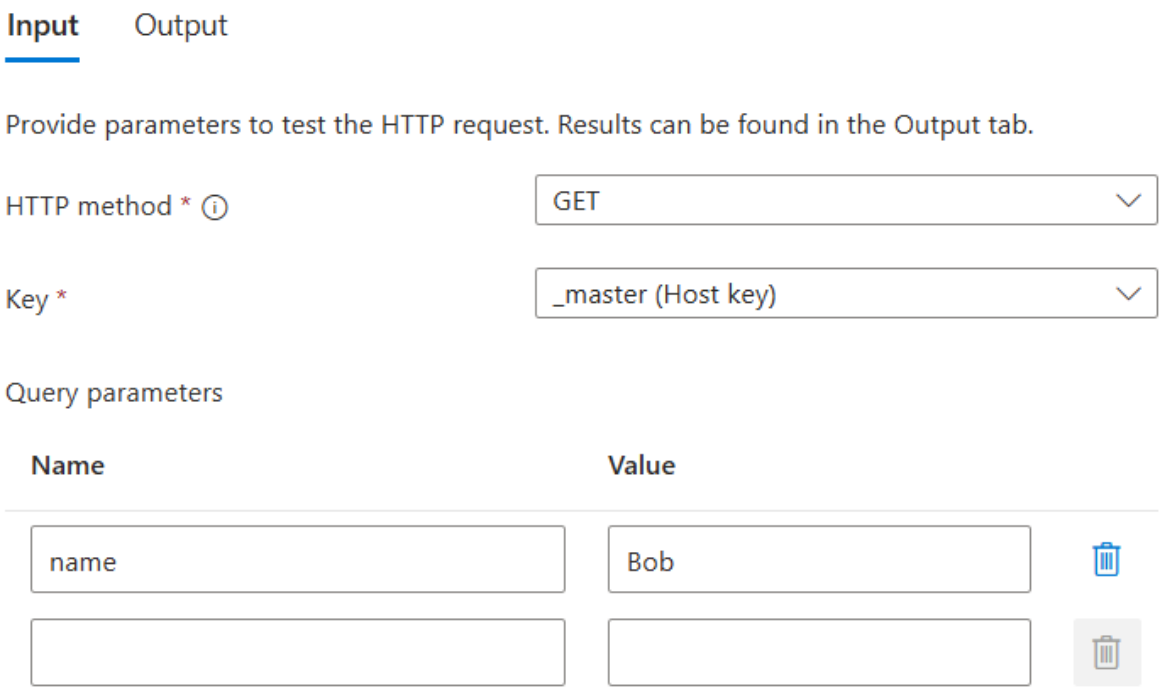
11. Now give a name to your function and then click on create.



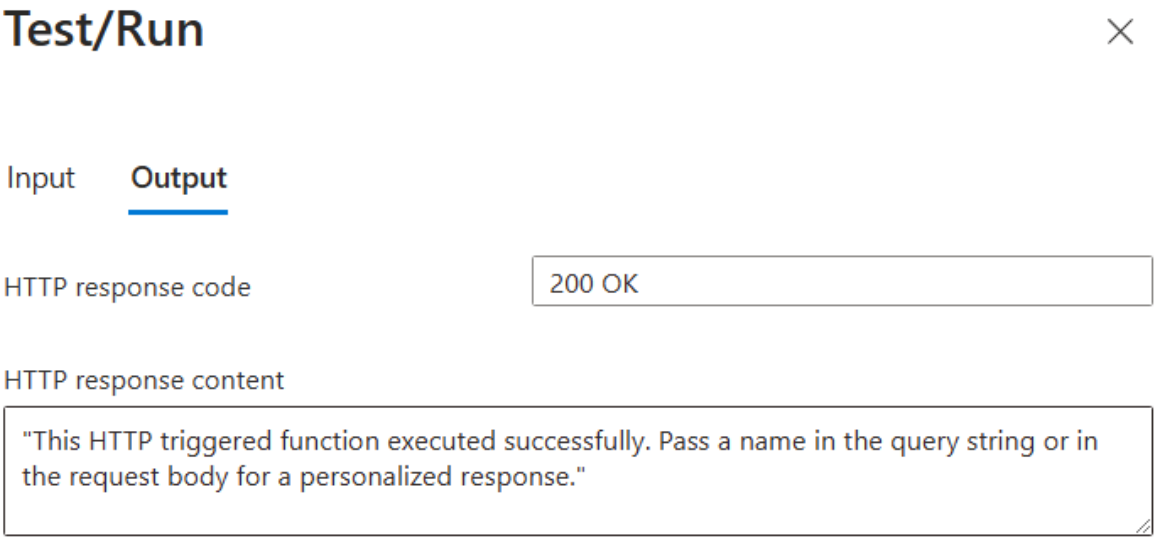
12. Once your function is created you will be directly inside it and you will get a default C# code. Click on Test/Run.



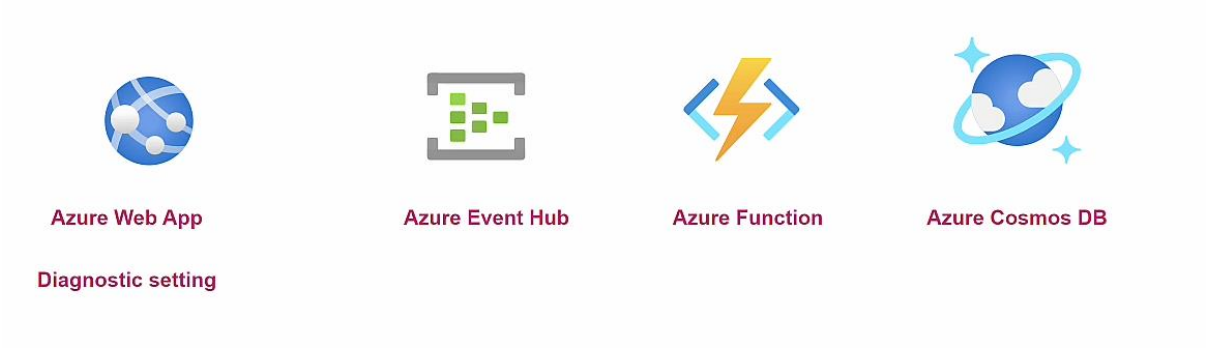
13. Then in the Input we need to use Get as our method then in the query parameters we can give a name and value.



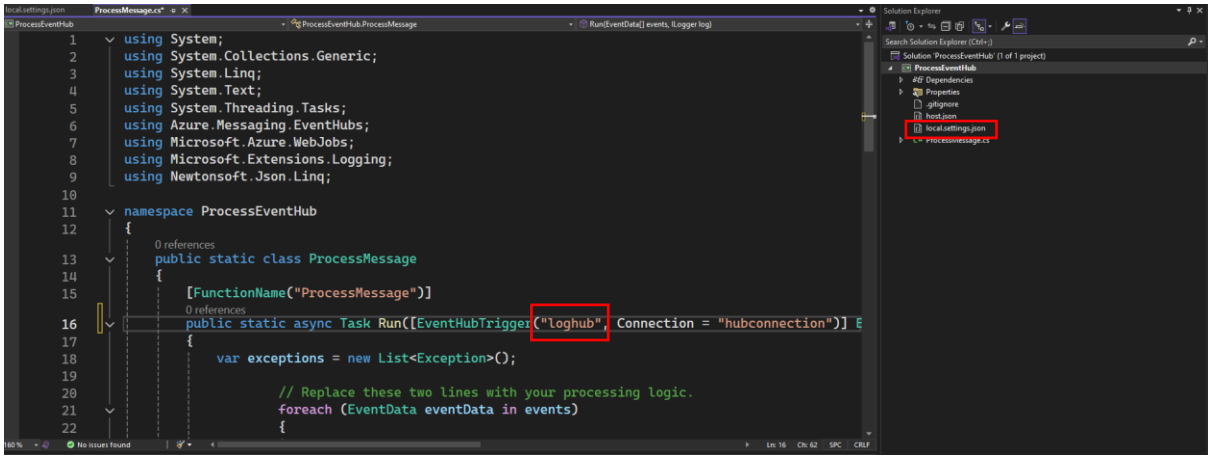
14. And in the output, you can get the status of 200 OK.



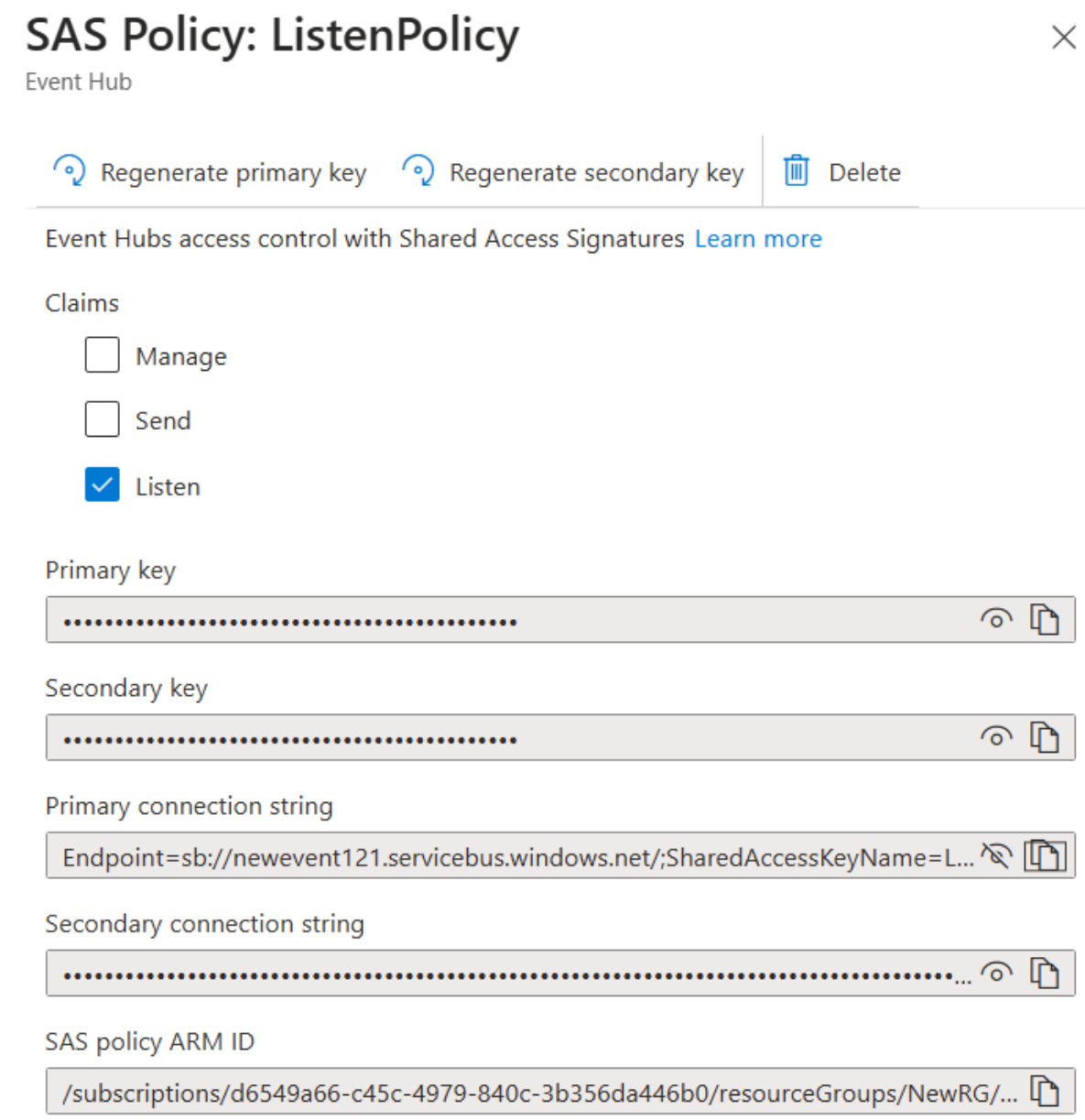
15. So, below is the structure diagram of what we will doing next. First, if you remember from the previous lab we are sending logs from the Azure web app to the Azure Event hubs.
16. Now we want the Azure function should consume the event from the event hub and then send it onto a container in Azure Cosmos DB.



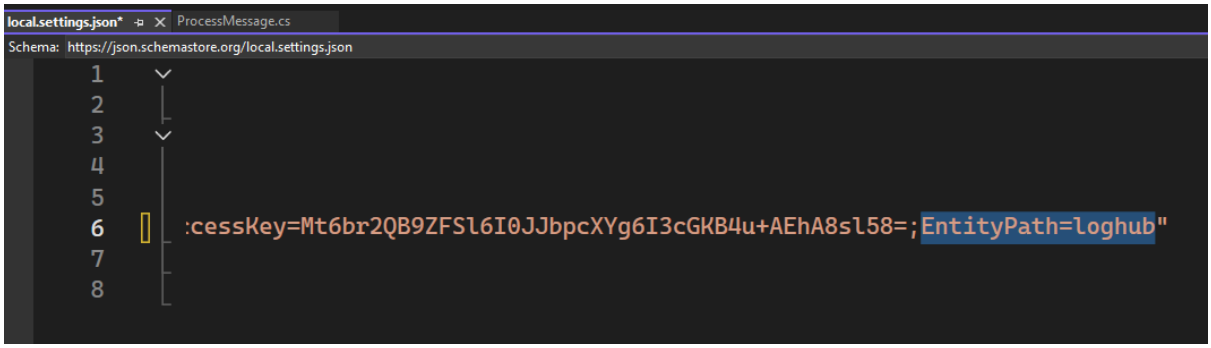
17. We also looked at the triggers that are available in Azure Function. Previously we created a trigger based on HTTP now we can create a trigger based on the Azure event hub.
18. But this time we have a program, so this is an Azure function program defined in Visual Studio that can go ahead and read the events in Azure Event hubs.
19. So, you will find two programs attached to this lab named **Part 1 and Part 2**. First, extract this **part 1 program** and open it in **Visual Studio 2022**.
20. In the program on line number 16 you need to change the name of the event hub with your event hub's name.
21. Then you need to move to the local settings JSON file to change the connection string.

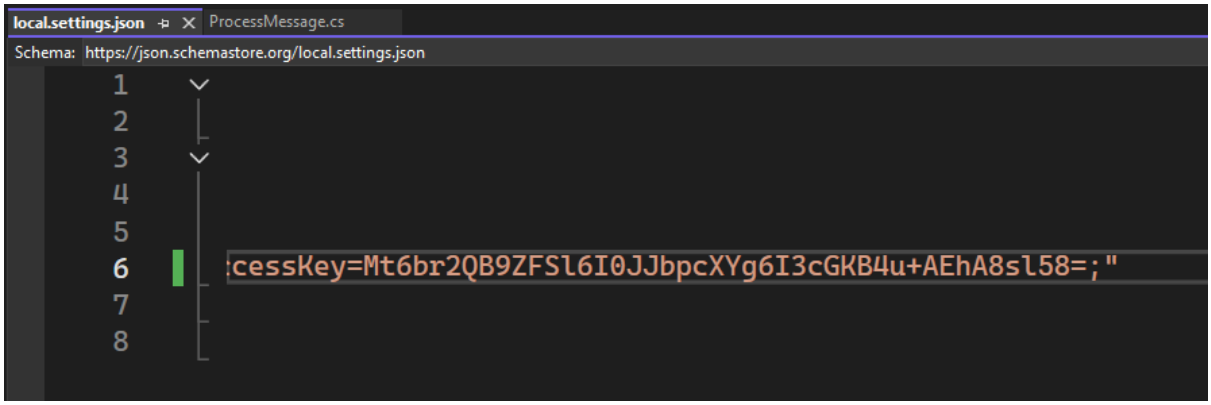


22. From the event hub go to the shared access policy and from your listen policy copy the connection string.

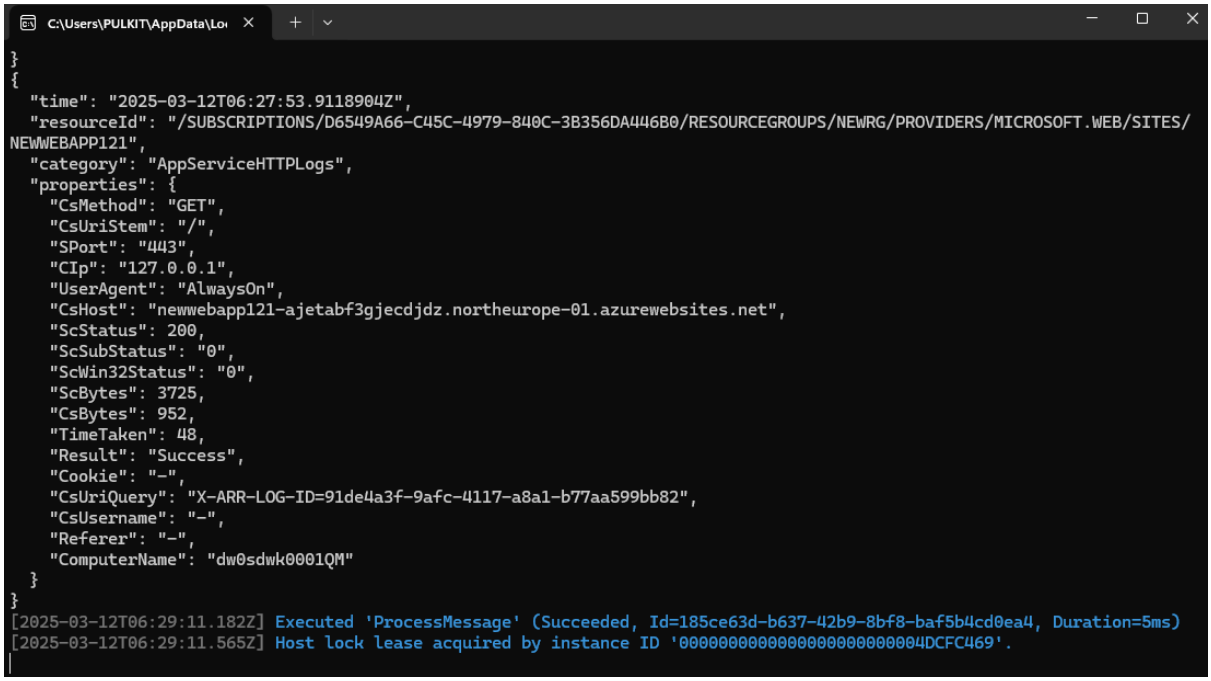


23. Then in the program paste it on line number 6 in the local.setting.json file. Also, remove the entity path from the connection string because we are already using the name.

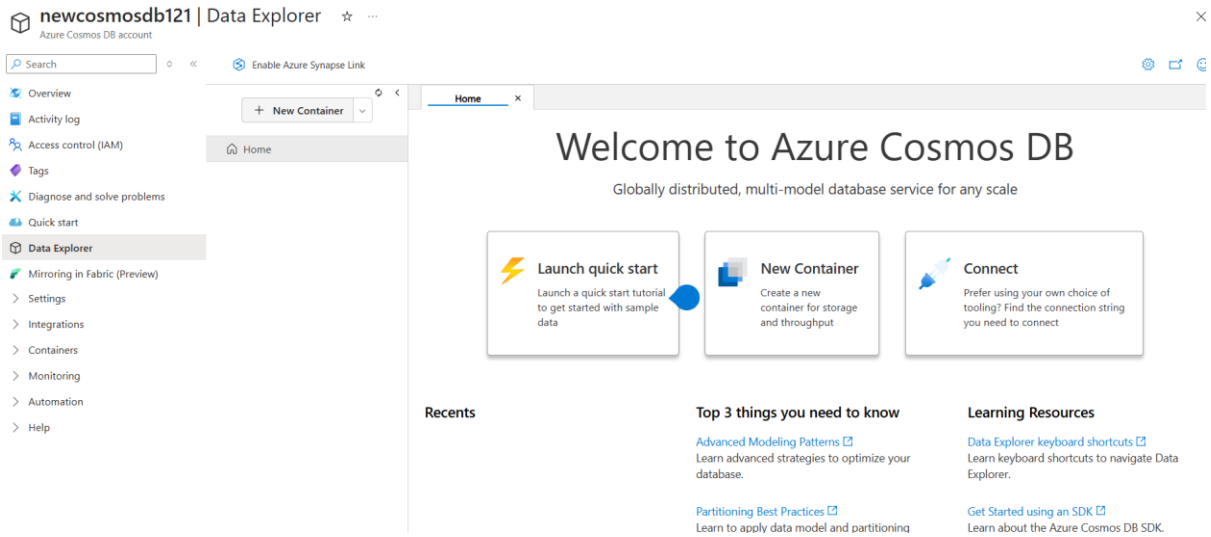




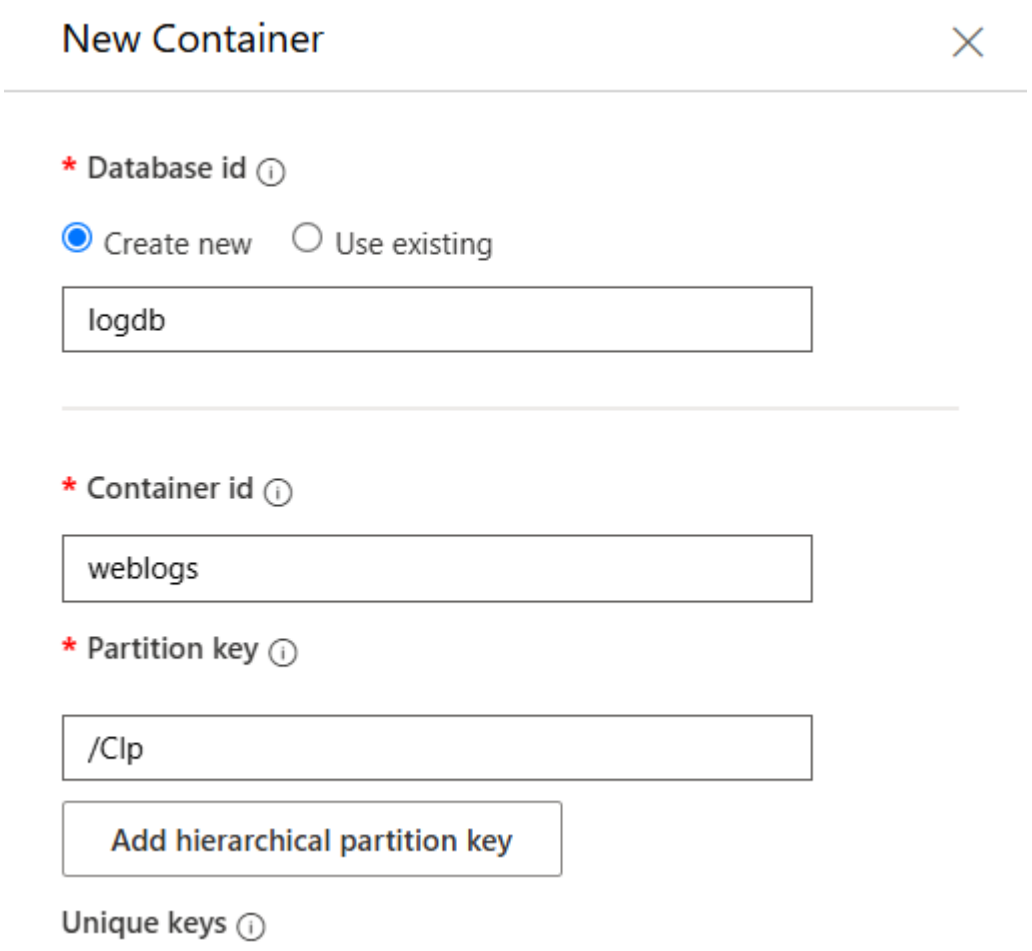
24. After saving everything just run the program locally. So, you will see that you are getting the information accordingly.



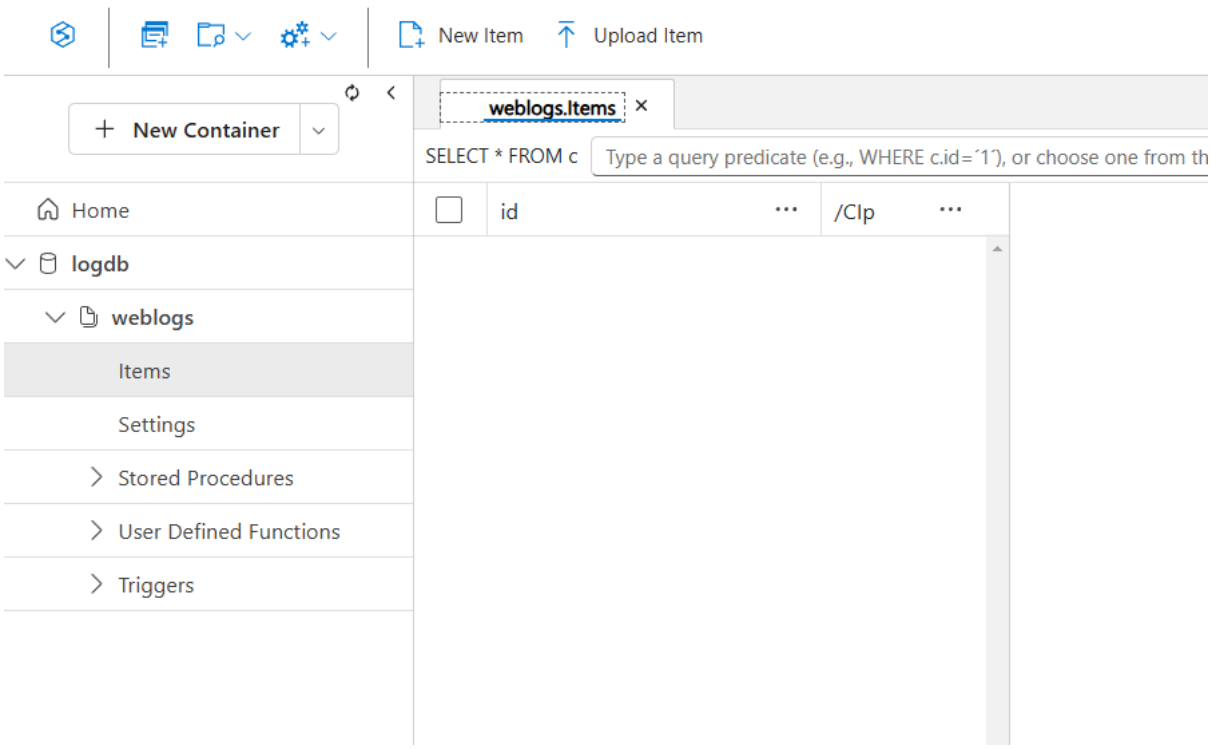
25. Now we are going to open the other program named Part 2. The code in this program will take each event and write them onto the Azure Cosmos DB.
26. We already have an Azure Cosmos DB account. Open it and go to Data Explorer. Then create a new container and DB.



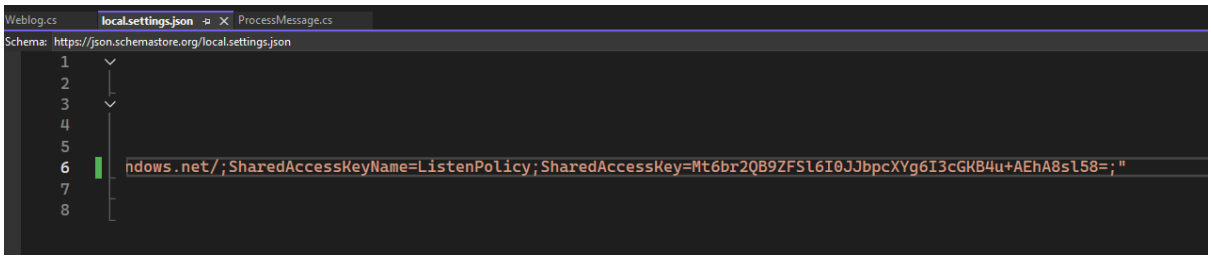
27. Then give a Database ID and Container ID. For the Partition key say CIp.



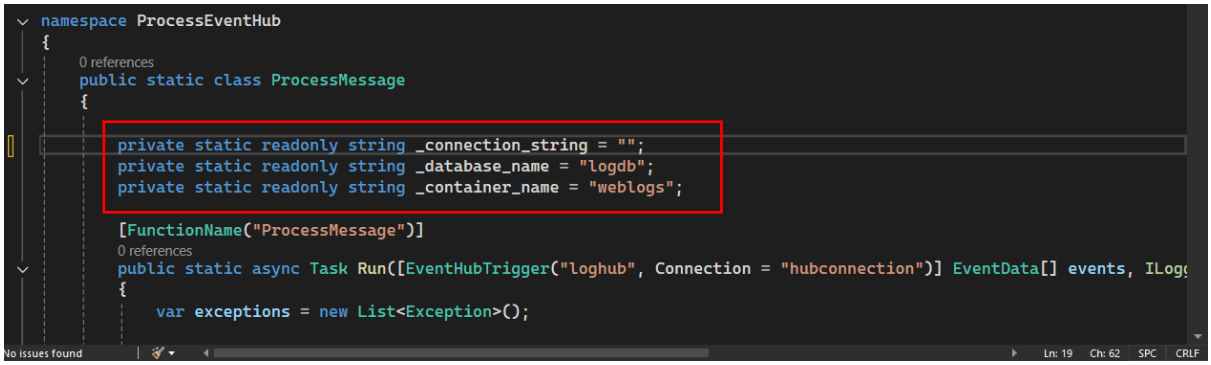
28. You will not have any items within the container, our Azure Function will actually go ahead and pick up the events from the Event Hub and then only take the data that is required and send it as individual items in the container in the logdb database.



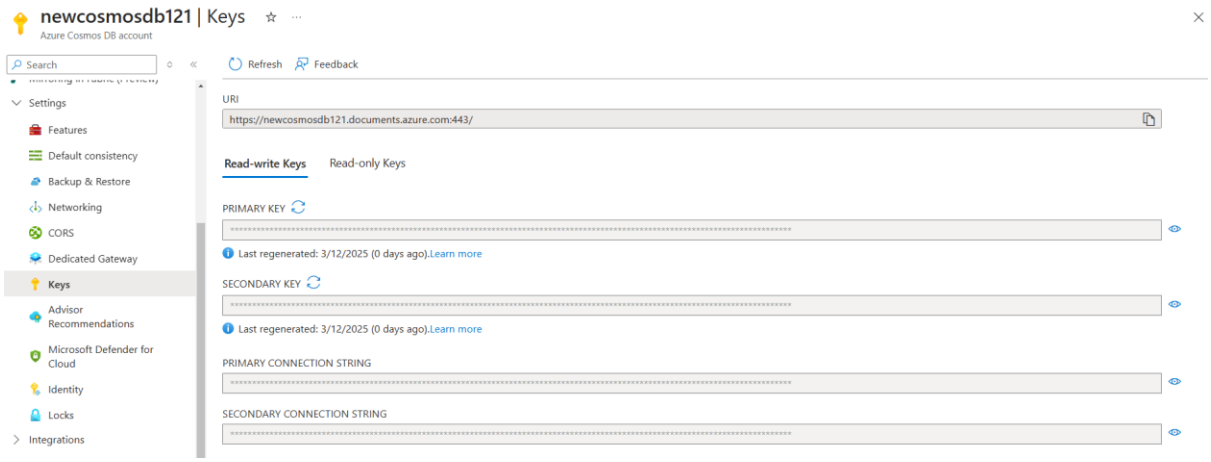
29. In the second program we also need to change the connection string and remove the entity path in the local.settings.json file.



30. Then we need to change the database name container name and give the connection string for the Cosmos DB.



31. In your cosmos DB account go to keys and copy the connection string from there.
32. Then just save everything and run the program.



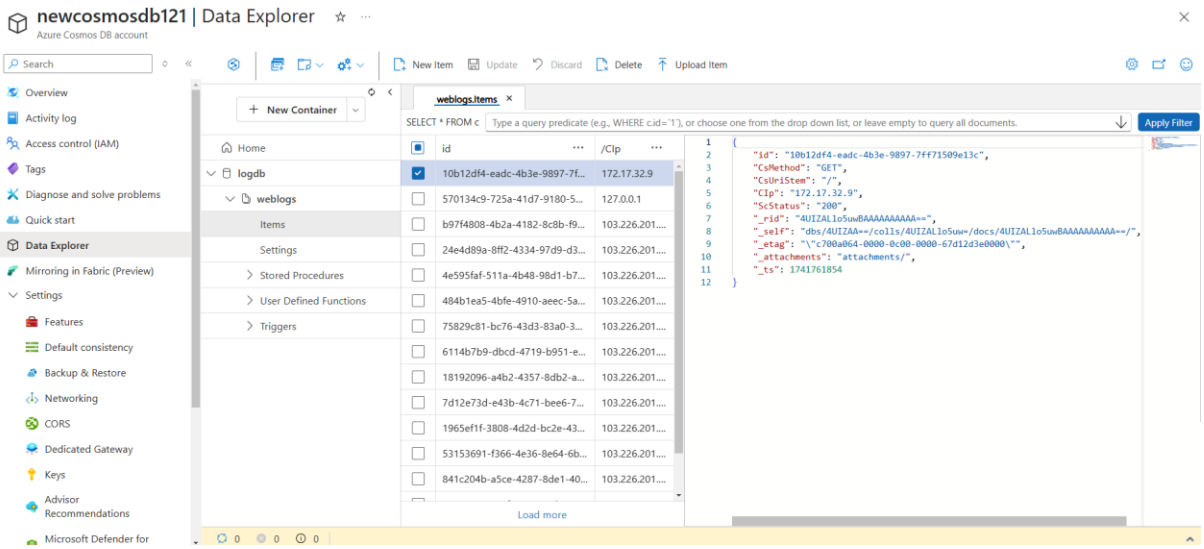

```
0 references
public static class ProcessMessage
{

    private static readonly string _connection_string = "AccountEndpoint=https://newcosmosdb121.documents.azure.com
    private static readonly string _database_name = "logdb";
    private static readonly string _container_name = "weblogs";

    [FunctionName("ProcessMessage")]
    0 references
    public static async Task Run([EventHubTrigger("loghub", Connection = "hubconnection")] EventData[] events, ILogg
    {
        var exceptions = new List<Exception>();
```

No issues found Ln: 19 Ch: 226 SPC CRLF

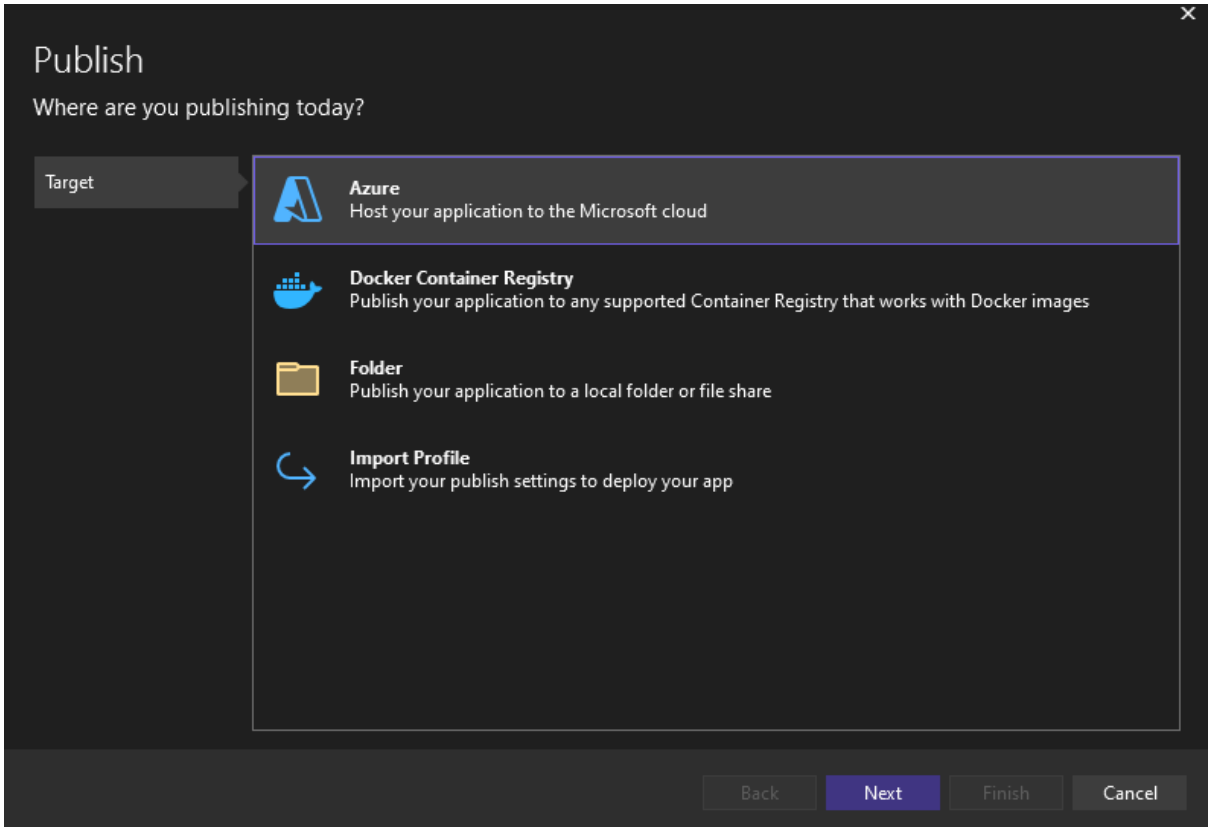
33. After a minute or so we can see that the item has been created and they are also added to the database.

```
Functions:

        ProcessMessage: eventHubTrigger

For detailed output, run func with --verbose flag.
[2025-03-12T06:42:46.275Z] Host lock lease acquired by instance ID '000000000000000000000004DCFC469'.
[2025-03-12T06:44:10.409Z] Executing 'ProcessMessage' (Reason='(null)', Id=bf76ff29-42c1-4fdd-a3e1-65bfc8e1b27c)
[2025-03-12T06:44:10.411Z] Trigger Details: PartionId: 0, Offset: 21536-21536, EnqueueTimeUtc: 2025-03-12T06:44:10.1650000+00:00-2025-03-12T06:44:10.1650000+00:00, SequenceNumber: 12-12, Count: 1
Item created
Item created
[2025-03-12T06:44:14.971Z] Executed 'ProcessMessage' (Succeeded, Id=bf76ff29-42c1-4fdd-a3e1-65bfc8e1b27c, Duration=4594ms)
[2025-03-12T06:46:11.890Z] Executing 'ProcessMessage' (Reason='(null)', Id=13dc17f3-ba0b-4f2b-8898-3d8a5093d0bb)
[2025-03-12T06:46:11.890Z] Trigger Details: PartionId: 1, Offset: 22624-22624, EnqueueTimeUtc: 2025-03-12T06:46:11.7180000+00:00-2025-03-12T06:46:11.7180000+00:00, SequenceNumber: 7-7, Count: 1
Item created
Item created
Item created
Item created
Item created
Item created
Item created
Item created
Item created
Item created
Item created
Item created
[2025-03-12T06:46:17.039Z] Executed 'ProcessMessage' (Succeeded, Id=13dc17f3-ba0b-4f2b-8898-3d8a5093d0bb, Duration=5149ms)
```

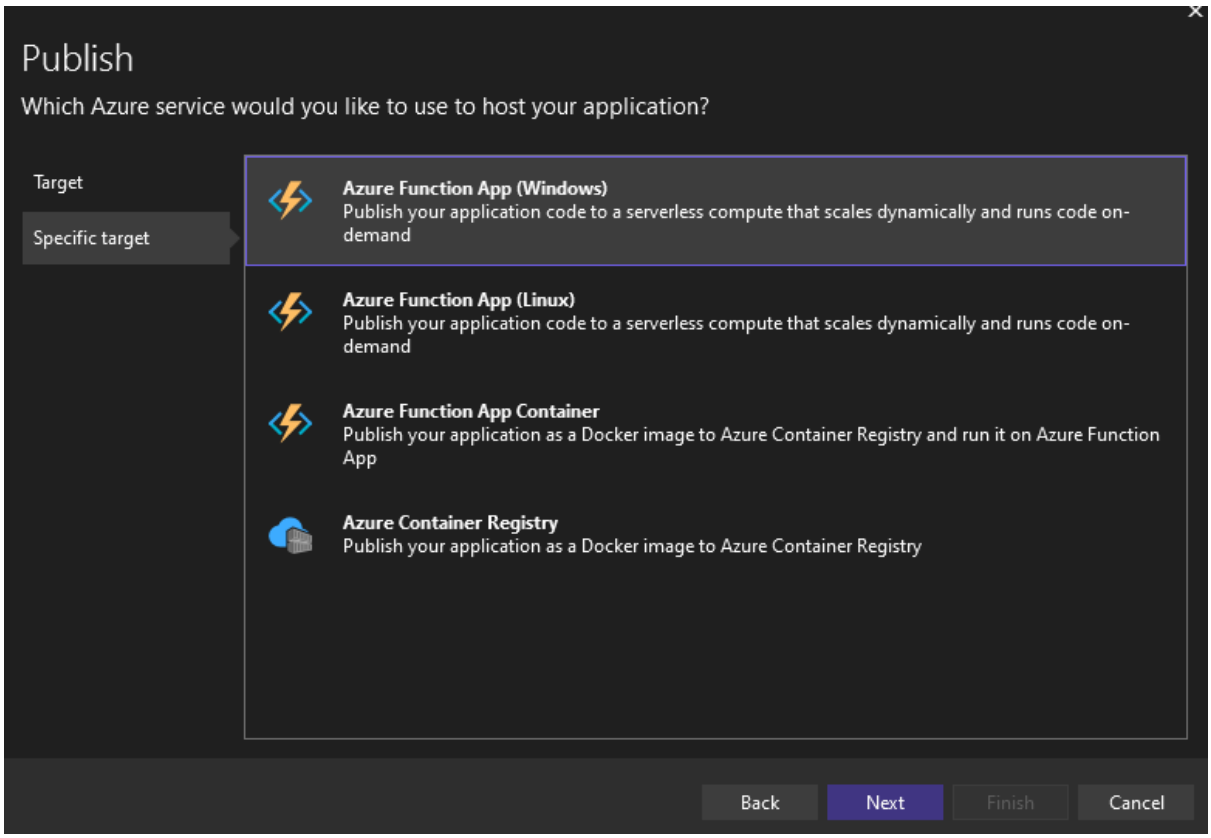
34. You can see that the items have been added. Currently, we have 15 items.



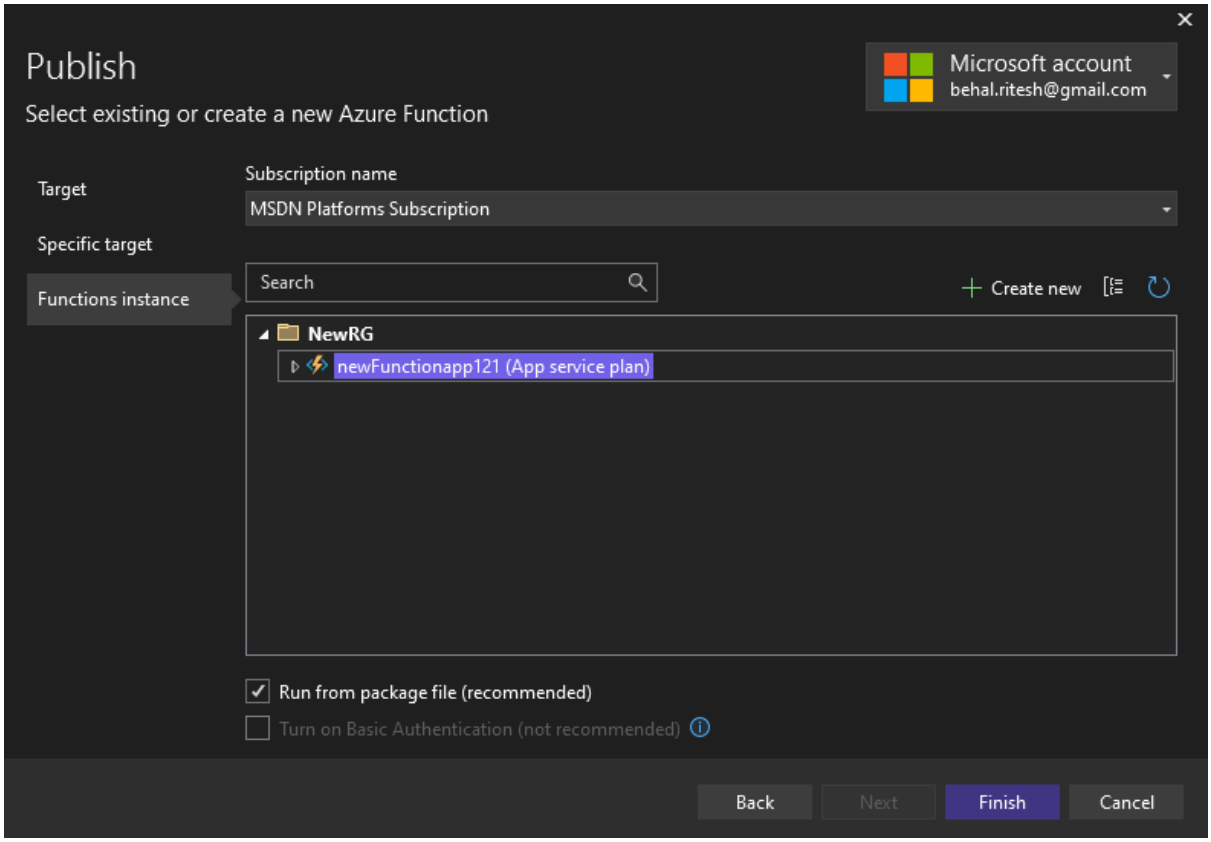
35. We can also publish our program to our Azure Function on the Portal. So, we have a function in place.
36. So, go back to the program right click on the project name from the solution explorer. Then chooser Azure click on next.



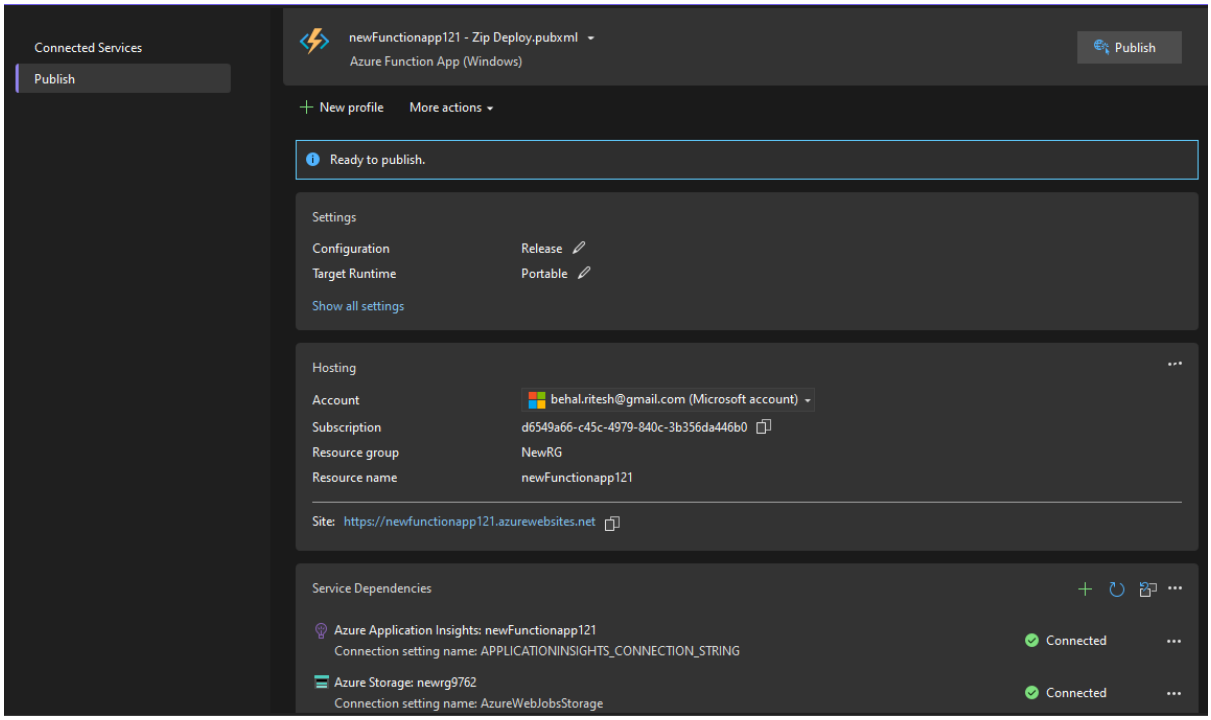
37. Choose the Azure function, and click on next/



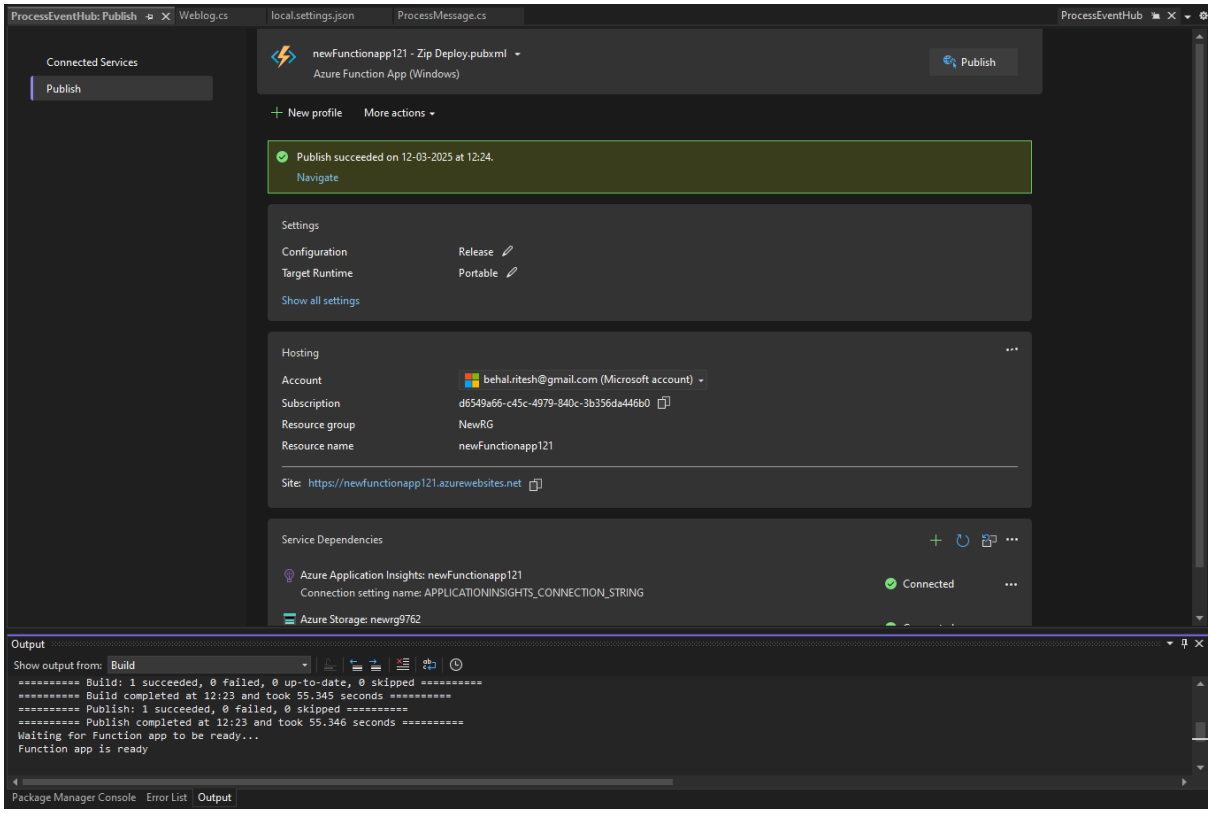
38. Choose your function from the resource group and click on finish.



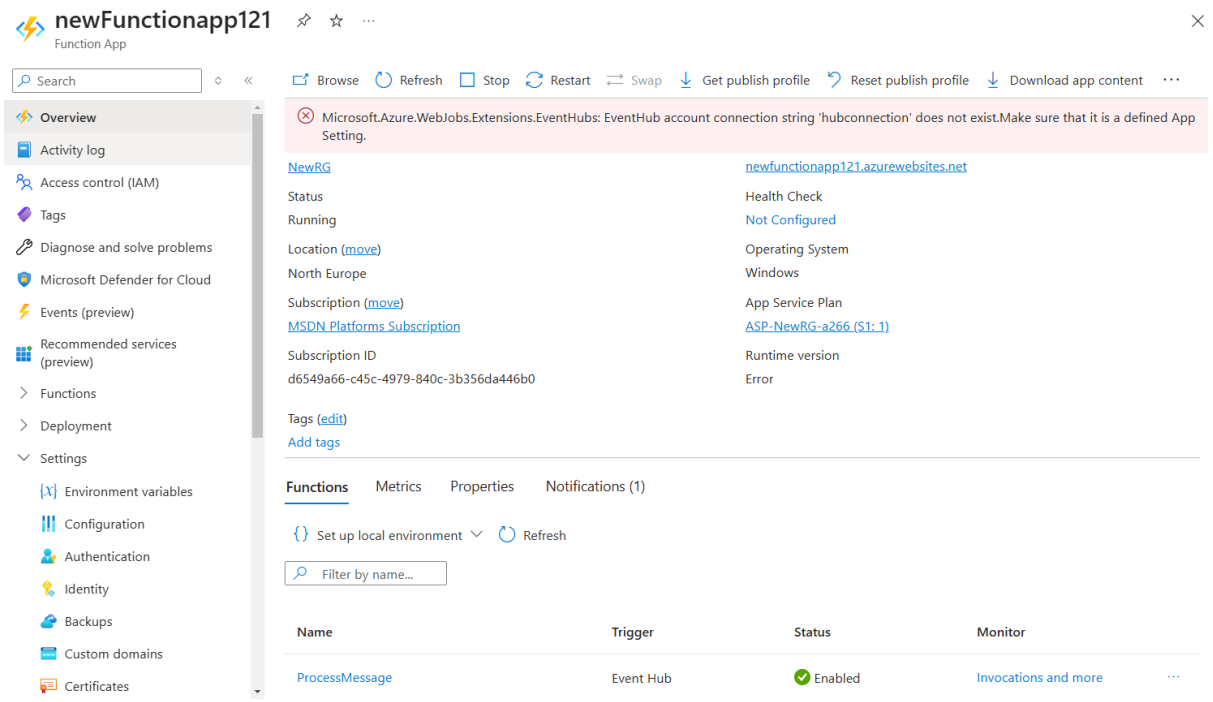
39. In the end click on publish and wait for the function to get ready.

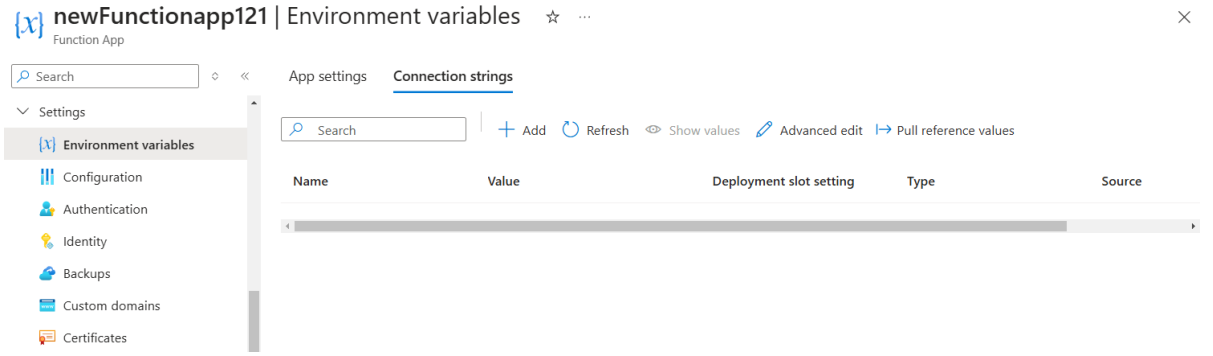


40. Here you can see that our function and we can view this in the Azure portal.

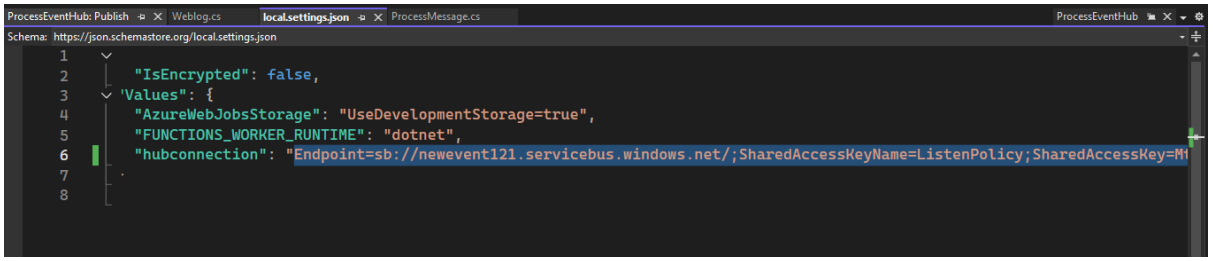


41. Our function is ready and enabled but we are getting an error that the hub connection does not exist. So, to remove this error we need to go to the environment variables tab in the function app. Click on add connection strings.

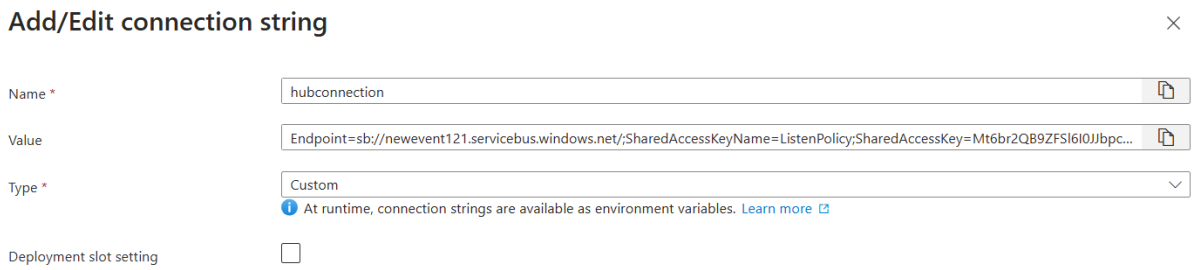




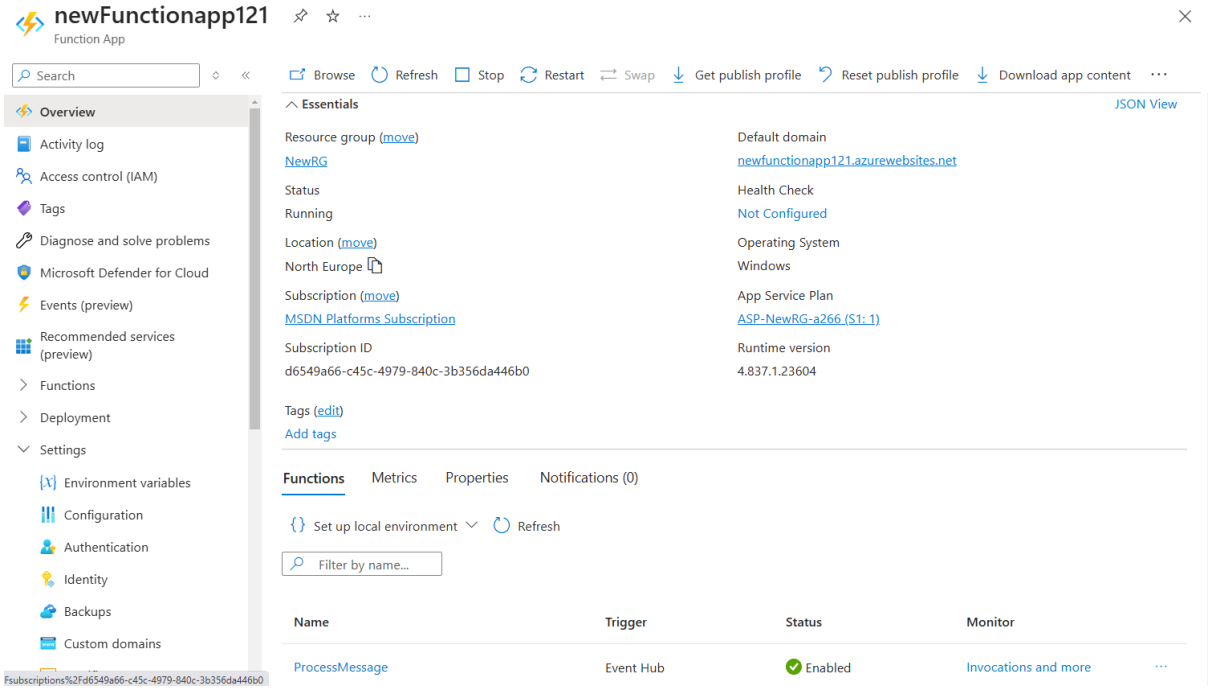
42. Now to get the connection strings we can either go to the event and copy the connection string from there or we can open our program and from the local.settings.json file, we can get the hub connection.



43. Now you need to give a name and, in the value, give the connection string then choose custom for the type. Click on apply.



44. What we can do now is, refresh our web app so that logs get generated and wait for some time.
45. As you can see below the error has been gone now



46. When we return to the items in the cosmos DB, we can see multiple items added here.